LAPORAN PRAKTIKUM

Teknologi IoT II

Modul 6: Protokol Komunikasi Telemetri Industri (MQTT)

DISUSUN OLEH:

Nama : Zahara Ummu Abdillah

NPM : 25782058

Kelas :3B

Dosen Pengampu:

Dr. Ir. Septafiansyah Dwi Putra

Agus Ambarwari, S.Pd., M.kom

TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

2026/2027

6.3 Teori Dasar

- 6.3.1 Arsitektur Publish-Subscribe (MQTT):

  Beroperasi menggunakan pemisahan ruang (*spatial decoupling*), waktu (*temporal decoupling*), dan sinkronisasi (*synchronization decoupling*) melalui perantara *MQTT Broker*.

- 6.3.2 Hierarki Topik (*Topic Hierarchy*):

  Pengalamatan pesan menggunakan string berjenjang (misalnya kampus/labiot/nim/telemetri) dengan dukungan karakter *wildcard* + (*single-level*) dan # (*multi-level*).

- 6.3.3 Tingkat Keandalan Pesan (QoS):

  Memiliki 3 tingkatan jaminan pengiriman, yaitu QoS 0 (*Fire and Forget*), QoS 1 (*At Least Once*), dan QoS 2 (*Exactly Once*).

- 6.3.4 Format Serialisasi Data (JSON):

  Menyatukan berbagai variabel telemetri ke dalam satu paket terstruktur berbasis pasangan *Key-Value* menggunakan pustaka ArduinoJson.

- 6.3.5 Fitur Keandalan (Keep-Alive & LWT):

  Menggunakan paket *PingREQ* untuk menjaga sesi TCP tetap hidup serta *Last Will and Testament* (LWT) untuk mengirim pesan darurat saat perangkat terputus secara tidak wajar.

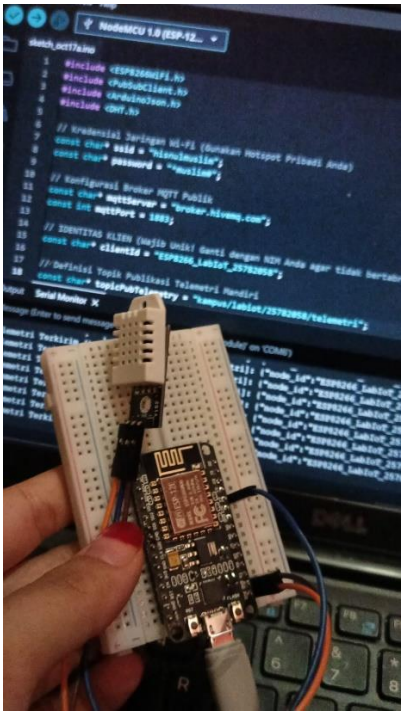
6.4 Persiapan Praktikum

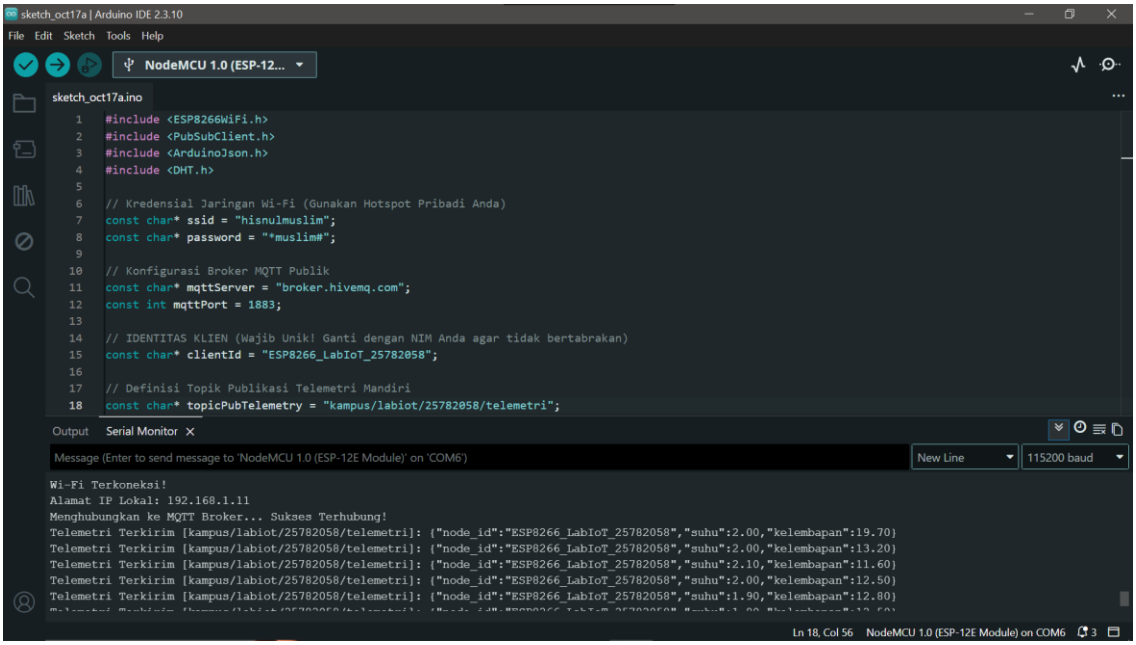
- 6.4.1 Kebutuhan Perangkat Keras: NodeMCU ESP8266, Sensor DHT11/DHT22, LED 5mm, Resistor $220\ \Omega$, kabel *jumper*, dan kabel data USB.
- 6.4.2 Persiapan Perangkat Lunak & Pustaka: Menginstal pustaka PubSubClient, ArduinoJson, DHT sensor library, serta menyiapkan aplikasi *MQTT Explorer* atau *HiveMQ Web Client*

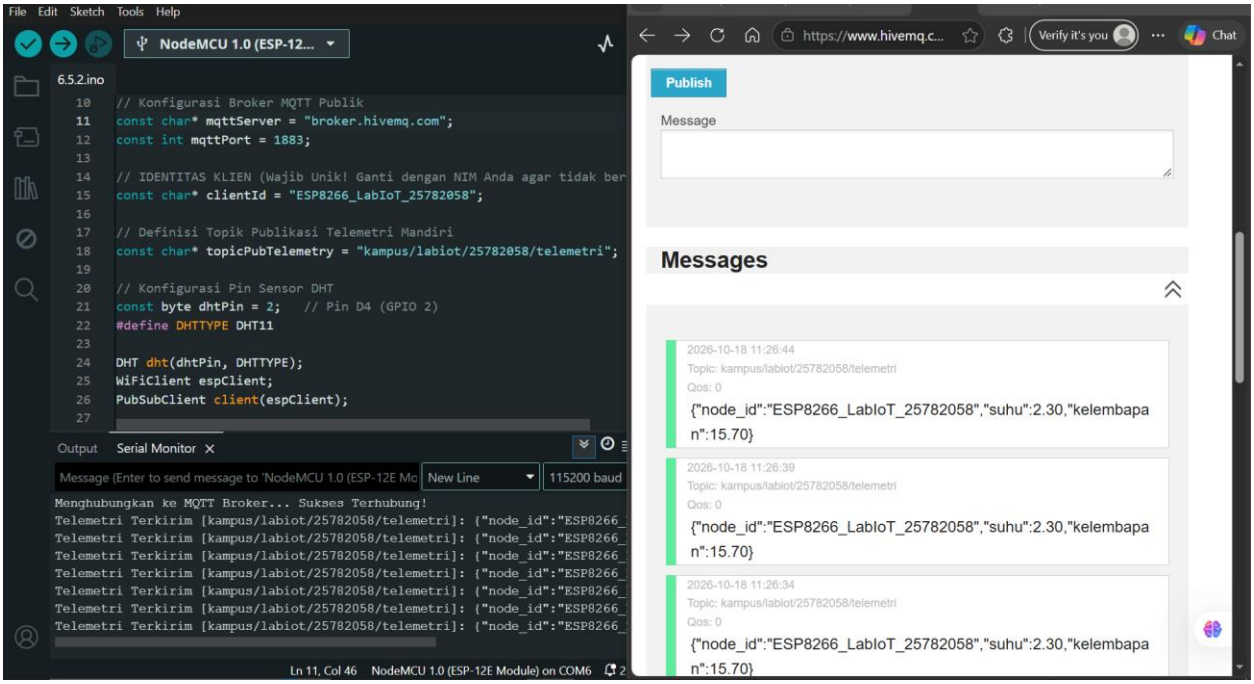
6.5 Praktikum

6.5.1 Praktikum 1: Publikasi Data Sensor Lingkungan (DHT) Berformat JSON

Fungsi Utama: NodeMCU membaca sensor DHT11/DHT22, mengemas datanya ke format JSON, lalu mempublikasikannya setiap 5 detik ke broker publik broker.hivemq.com.

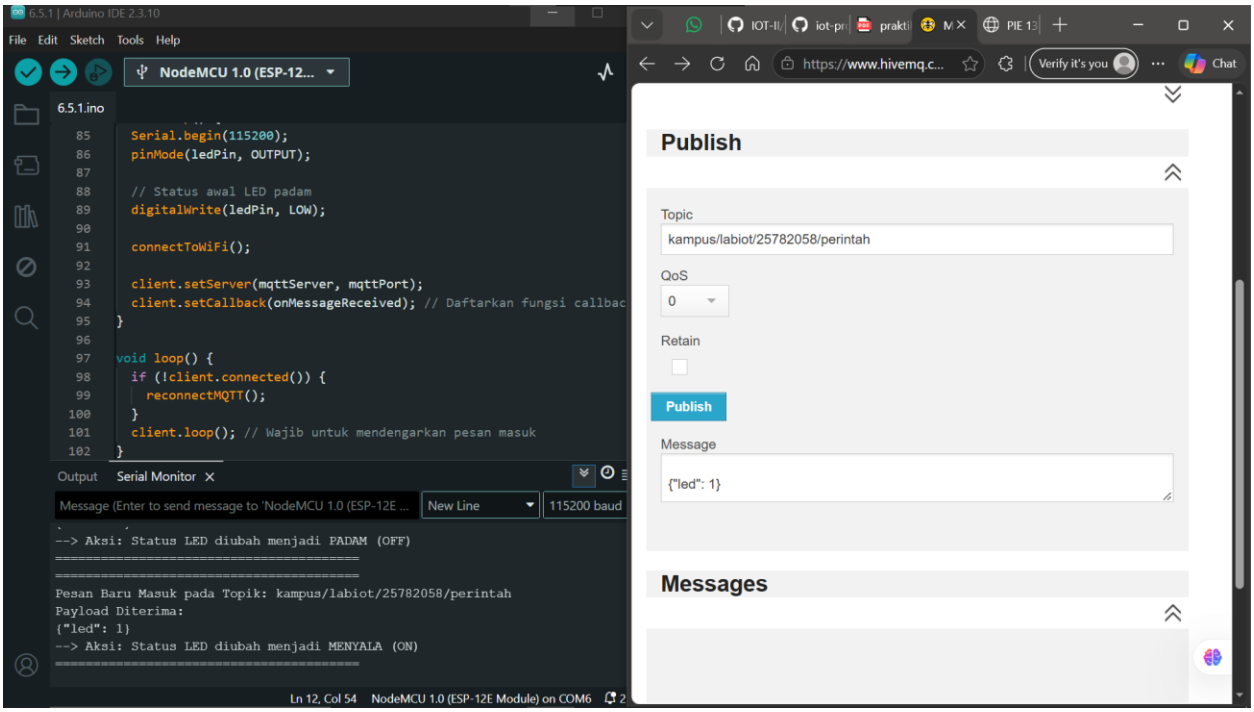


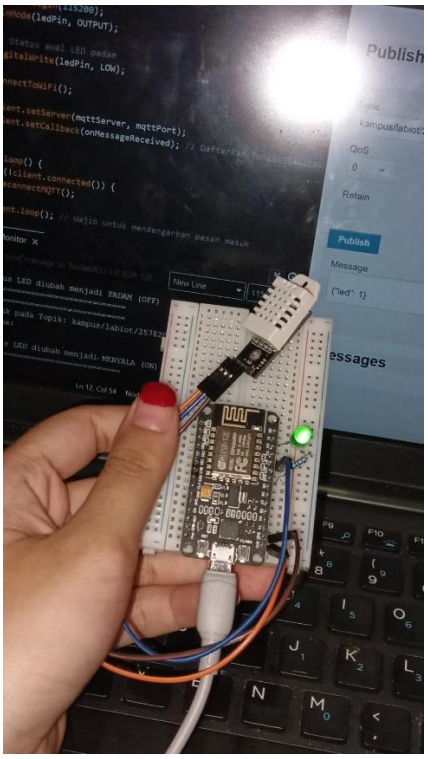


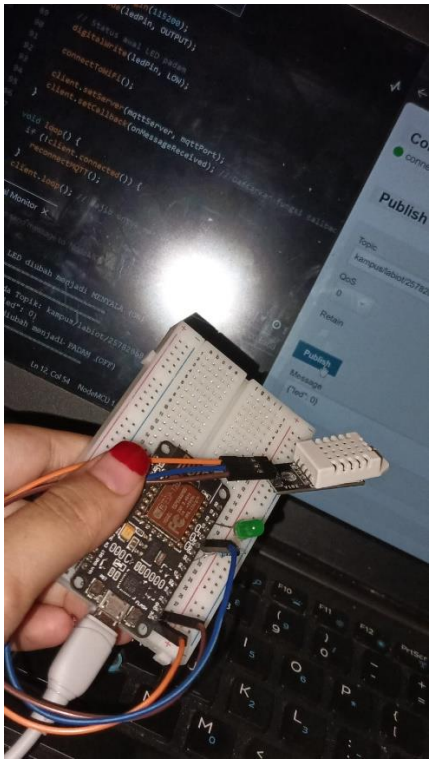

File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
6.5.2.ino
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
// IDENTITAS KLIEN (Wajib Unik! Ganti dengan NIM Anda agar tidak ber
const char* clientId = "ESP8266_LabIoT_25782058";
// Definisi Topik Publikasi Telemetri Mandiri
const char* topicPubTelemetry = "kampus/labiot/25782058/telemetri";
// Konfigurasi Pin Sensor DHT
const byte dhtPin = 2; // Pin D4 (GPIO 2)
#define DHTTYPE DHT11
DHT dht(dhtPin, DHTTYPE);
WiFiClient espClient;
PubSubClient client(espClient);
Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Mo
New Line
115200 baud
Menghubungkan ke MQTT Broker... Sukses Terhubung!
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Telemetri Terkirim [kampus/labiot/25782058/telemetri]: {"node_id":"ESP8266_
Ln 11, Col 46 NodeMCU 1.0 (ESP-12E Module) on COM6
https://www.hivemq.c...
Verify it's you
Chat
Publish
Message
Messages
2026-10-18 11:26:44
Topic: kampus/labiot/25782058/telemetri
Qos: 0
{"node_id":"ESP8266_LabIoT_25782058","suhu":2.30,"kelembapan":15.70}
2026-10-18 11:26:39
Topic: kampus/labiot/25782058/telemetri
Qos: 0
{"node_id":"ESP8266_LabIoT_25782058","suhu":2.30,"kelembapan":15.70}
2026-10-18 11:26:34
Topic: kampus/labiot/25782058/telemetri
Qos: 0
{"node_id":"ESP8266_LabIoT_25782058","suhu":2.30,"kelembapan":15.70}

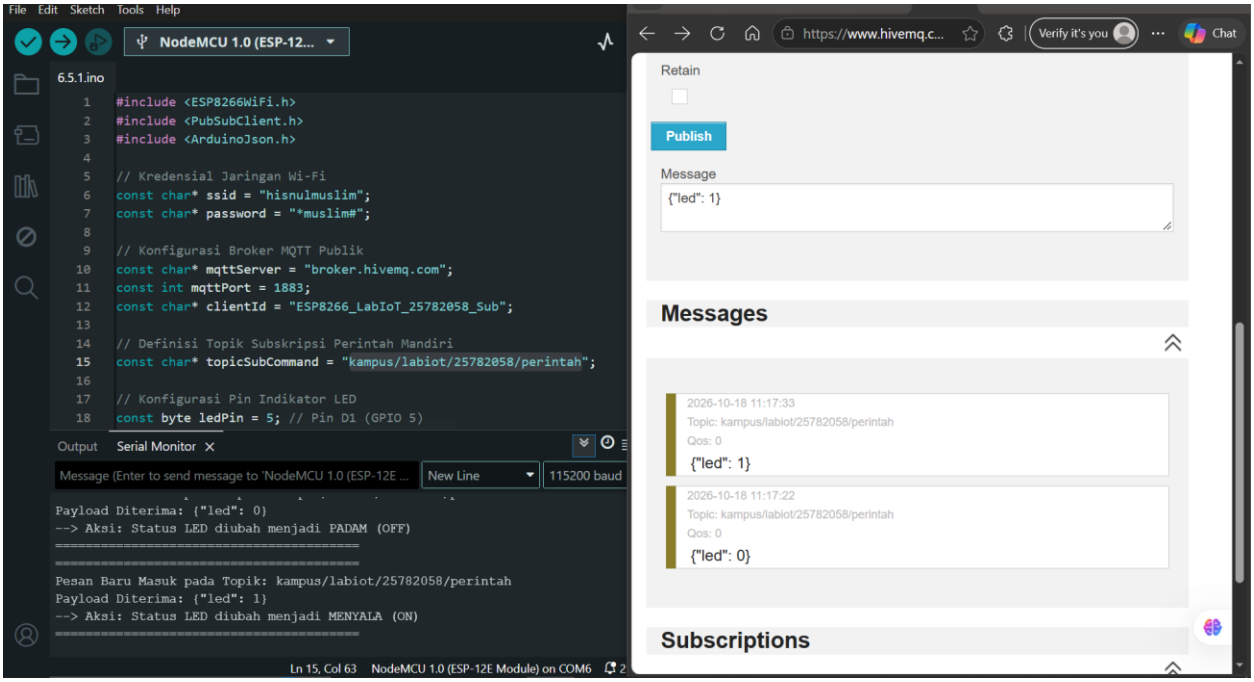
6.5.2 Praktikum 2: Subskripsi dan Kontrol Indikator LED Asinkron

Fungsi Utama: Memprogram NodeMCU sebagai Subscriber yang mendaftarkan diri pada topik perintah (kampus/labiot/25782058/perintah) untuk mengontrol LED secara asinkron.



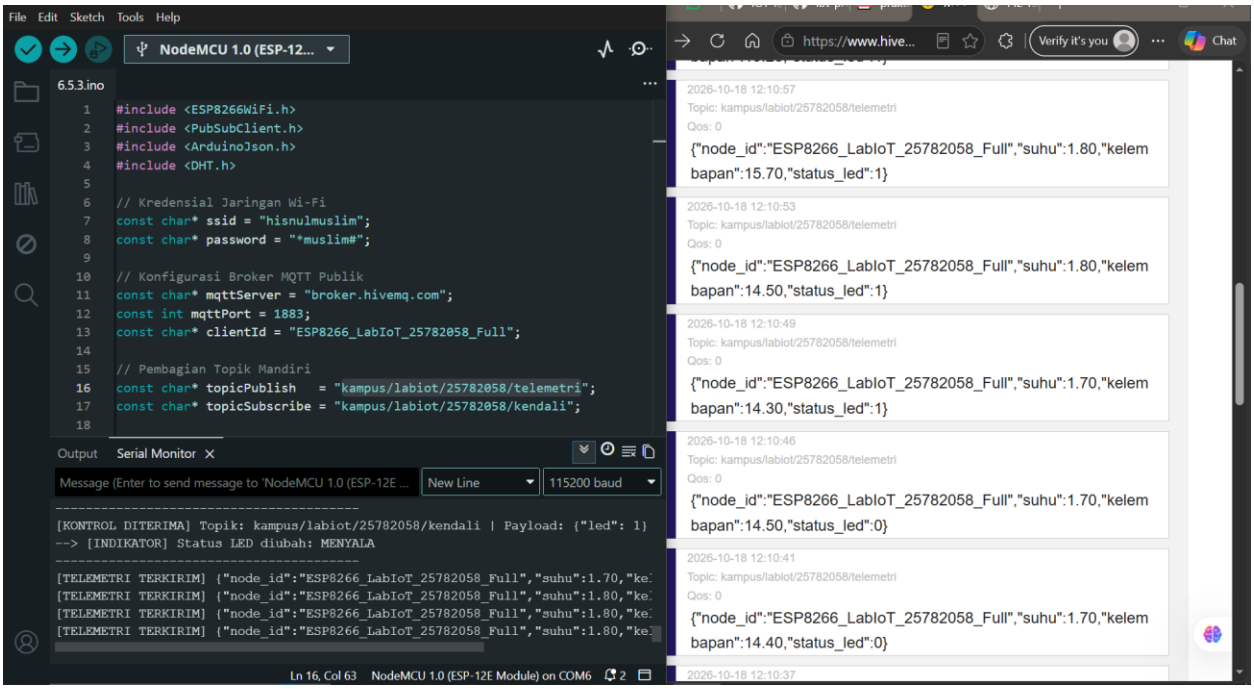


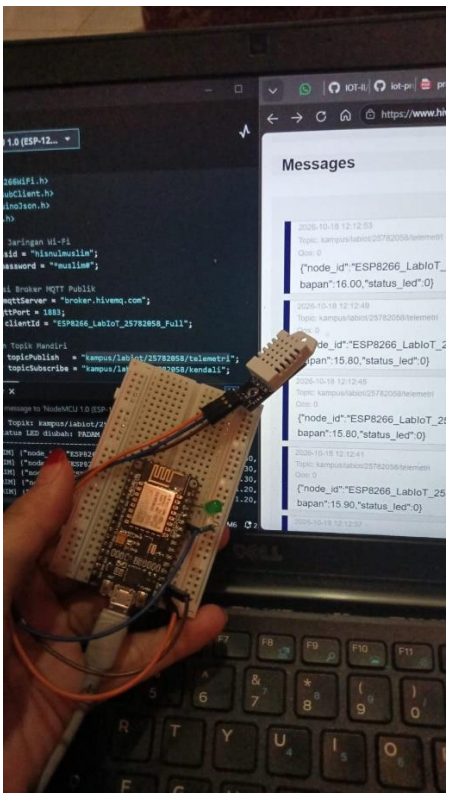


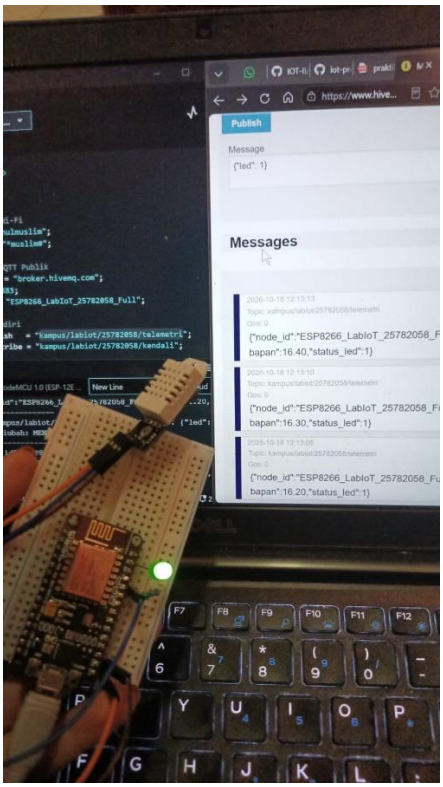

File Edit Sketch Tools Help
NodeMCU 1.0 (ESP-12...
6.5.1.ino
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
// Kredensial Jaringan Wi-Fi
const char* ssid = "hisnulmuslim";
const char* password = "*muslim#";
// Konfigurasi Broker MQTT Publik
const char* mqttServer = "broker.hivemq.com";
const int mqttPort = 1883;
const char* clientId = "ESP8266_LabIoT_25782058_Sub";
// Definisi Topik Subskripsi Perintah Mandiri
const char* topicSubCommand = "kampus/labiot/25782058/perintah";
// Konfigurasi Pin Indikator LED
const byte ledPin = 5; // Pin D1 (GPIO 5)
Output
Serial Monitor
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E ...
New Line
115200 baud
Payload Diterima: {"led": 0}
--> Aksi: Status LED diubah menjadi PADAM (OFF)
Pesan Baru Masuk pada Topik: kampus/labiot/25782058/perintah
Payload Diterima: {"led": 1}
--> Aksi: Status LED diubah menjadi MENYALA (ON)
Ln 15, Col 63
NodeMCU 1.0 (ESP-12E Module) on COM6
https://www.hivemq.c...
Verify it's you
Chat
Retain
Publish
Message
{"led": 1}
Messages
2026-10-18 11:17:33
Topic: kampus/labiot/25782058/perintah
Qos: 0
{"led": 1}
2026-10-18 11:17:22
Topic: kampus/labiot/25782058/perintah
Qos: 0
{"led": 0}
Subscriptions

6.5.3 Praktikum 3: Sistem Terpadu Dua Arah (Full Duplex Telemetry & Control)

Fungsi Utama: Menggabungkan pengiriman telemetri periodik dan fungsi *listener* kendali LED secara simultan (*Full-Duplex*).







6.6 Latihan

Latihan 1 (Analisis Arsitektur & Perbandingan Protokol):

1. **Efisiensi Penggunaan Bandwidth pada Paket Data Seluler:**
   - Protokol HTTP/REST berbasis *Request-Response* memerlukan pengiriman header HTTP yang berukuran sangat besar ($\approx 200 - 800\ \text{byte}$) untuk setiap kali pertukaran data (baik saat meminta maupun mengirim data).
   - Sebaliknya, protokol MQTT dirancang dengan struktur header yang sangat minimalis (mulai dari 2 byte saja). Hal ini membuat ukuran paket data menjadi sangat kecil, sehingga jauh lebih hemat dan efisien dalam penggunaan *bandwidth* serta kuota data seluler, terutama pada jaringan IoT skala besar dengan perangkat yang mengirim data secara terus-menerus.
2. **Beban Komputasi Mikrokontroler (CPU Overhead) Saat Menangani Pertukaran Data Dua Arah:**
   - Pada HTTP, setiap kali klien ingin mengirim atau mengambil data, perangkat harus membuka dan menutup koneksi TCP baru secara berulang-ulang (*stateless*), yang membebani memori dan siklus prosesor mikrokontroler.
   - MQTT menggunakan koneksi TCP yang dipelihara secara berkelanjutan (*persistent connection*) dan dipantau secara ringan menggunakan sinyal *Keep-Alive Ping*. Mikrokontroler tidak perlu berulang kali melakukan *handshake* koneksi dari awal, sehingga beban pemrosesan (*CPU overhead*) menjadi sangat rendah dan sumber daya mikrokontroler dapat difokuskan untuk tugas utama sensor.
3. **Kemampuan Pengiriman Data Secara Simultan ke Banyak Perangkat Penerima (1-to Distribution):**
   - HTTP menggunakan model komunikasi *point-to-point*, di mana pelamar/peladen (*server*) harus melayani setiap permintaan dari banyak klien secara terpisah, sehingga sulit melakukan siaran massal secara efisien.
   - MQTT menggunakan model arsitektur terpusat Publish-Subscribe melalui perantara MQTT Broker. Dengan model ini, satu pengirim (*Publisher*) cukup memublikasikan data satu kali ke sebuah topik di *broker*, dan *broker* akan secara otomatis mendistribusikan data tersebut secara simultan ke seluruh perangkat penerima (*Subscriber*) yang berlangganan pada topik yang sama tanpa batasan jumlah perangkat.

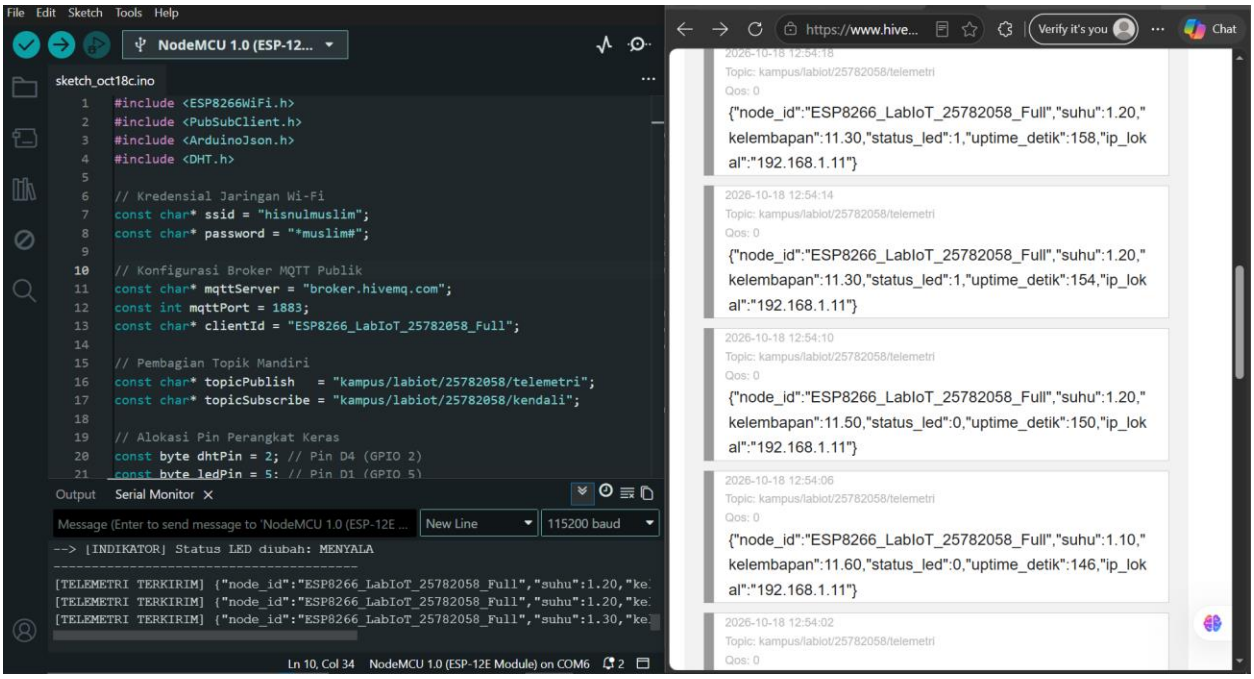
Latihan 2 (Analisis Hierarki Topik & Karakter Wildcard):

- Memantau suhu seluruh mesin: pabrik/+/+/suhu
- Memantau sensor mesin m1 di line2: pabrik/line2/m1/#
- Memantau seluruh lalu lintas pabrik: pabrik/#

Latihan 3 (Implementasi Serialisasi Data Dinamis):

- Modifikasi blok penyusunan JSON pada Praktikum 1 agar paket telemetri menyertakan dua parameter tambahan berikut:
  i. "uptime_detik": Lama waktu aktif sistem dalam satuan detik (dihitung dari rumus millis() / 1000).
  ii. "ip_lokal": Alamat IP lokal mikrokontroler dalam format string (WiFi.localIP().toString()).
  Tuliskan baris modifikasi kode C++ yang Anda tambahkan pada fungsi pembentukan JSON!





2026-10-18 12:54:14 Topic: kampus/labiot/25782058/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782058_Full","suhu":1.20,"kelembapan":11.30,"status_led":1,"uptime_detik":154,"ip_lokal":"192.168.1.11"}
2026-10-18 12:54:10 Topic: kampus/labiot/25782058/telem... Qos: 0
{"node_id":"ESP8266_LabIoT_25782058_Full","suhu":1.20,"kelembapan":11.50,"status_led":0,"uptime_detik":150,"ip_lokal":"192.168.1.11"}

Latihan 4 (Analisis Mekanisme Reconnect & Loop Blocking):

Pada Praktikum 3, di dalam fungsi loop() terdapat pemanggilan rutin client.loop().

Konsekuensi teknis jika menyisipkan instruksi delay(6000) di dalam fungsi loop():

- Mikrokontroler akan berhenti mengeksekusi rutin client.loop() selama 6 detik penuh.
- Akibatnya, hubungan *keep-alive ping* dengan *broker* akan terputus karena terlewat dari batas waktu, pesan *callback* untuk penanganan kendali LED tidak akan direspons atau mengalami *lag* yang parah, dan sistem menjadi tidak stabil hingga berisiko diputus paksa oleh *broker* akibat *timeout*.

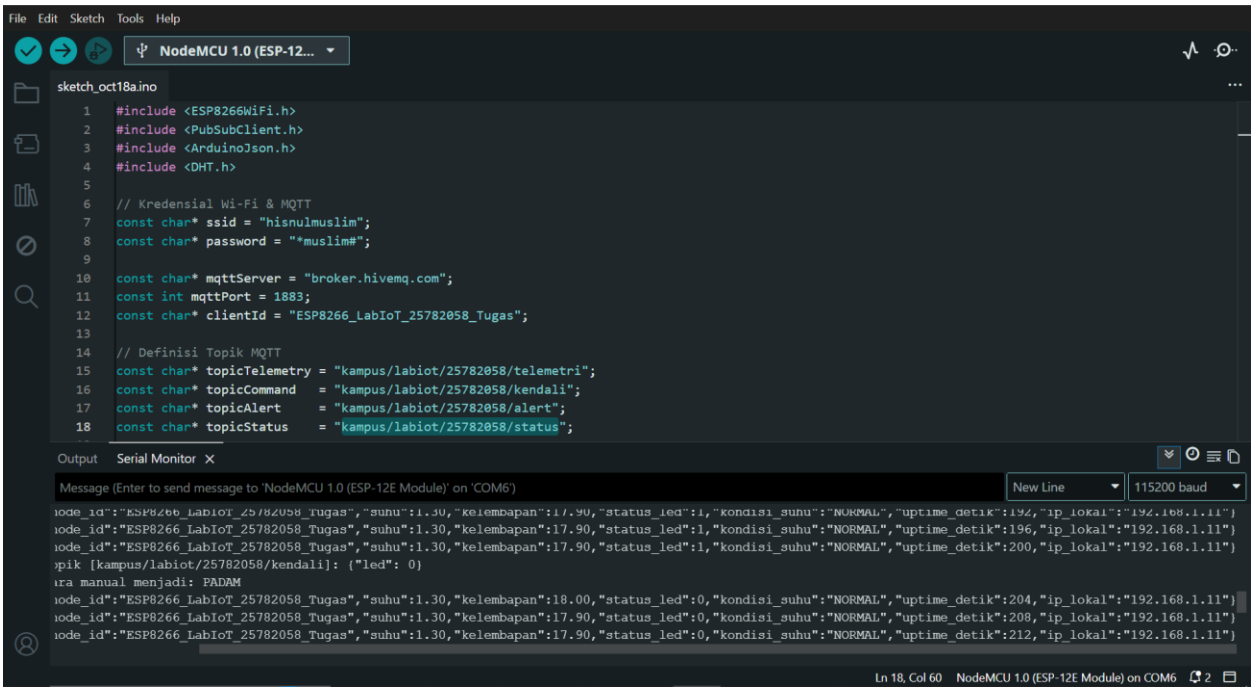
Cara kerja fungsi reconnect() memulihkan koneksi:

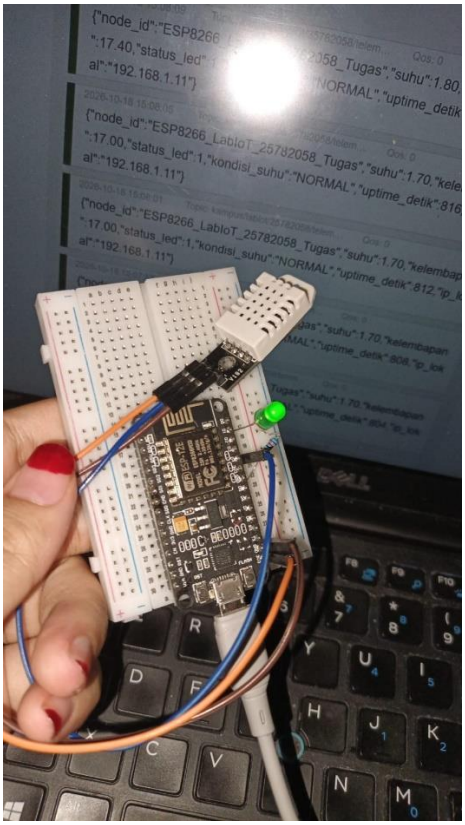
- Fungsi reconnect() bekerja dengan memantau status koneksi secara kontinu menggunakan kondisi pengecekan !client.connected().
- Jika terdeteksi bahwa mikrokontroler terputus dari jaringan, fungsi ini akan menjalankan perulangan untuk mencoba menghubungkan kembali klien ke *broker* MQTT melalui client.connect(clientId).
- Sesaat setelah koneksi berhasil pulih (*terhubung kembali*), fungsi ini akan langsung mendaftarkan ulang subskripsi topik yang diperlukan menggunakan client.subscribe(...) agar perangkat dapat kembali menerima data atau perintah dari luar secara normal.

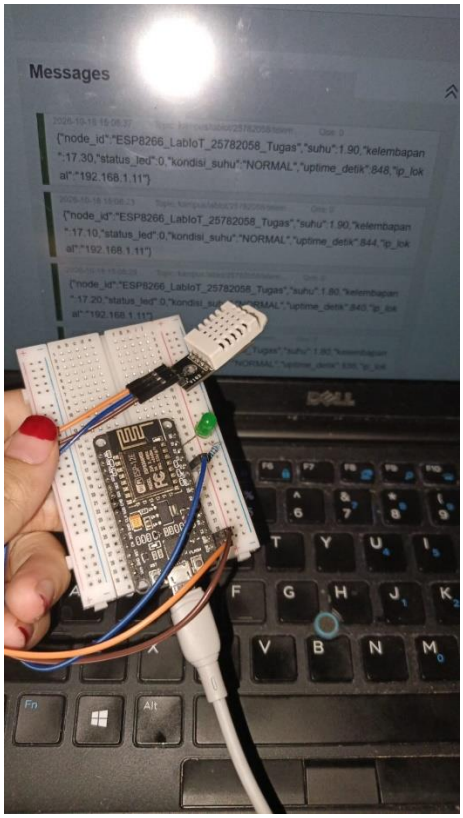
6.7 Tugas: Sistem Telemetri & Kendali Closed-Loop Industri

Fungsi Utama: Membangun sistem keselamatan otomatis (*Edge Intelligence*) di mana suhu $> 31.0^\circ\text{C}$ akan memicu status "PANAS", mengirim *alert* darurat otomatis ke topik .../alert, menyalakan LED darurat, serta mendukung fitur *Last Will and Testament* (LWT) dan *Manual Override*.









6.8 Rangkuman

Pada Modul 6 ini, kita telah melangkah ke standar komunikasi industri modern dengan menguasai protokol MQTT (Message Queuing Telemetry Transport). Melalui arsitektur terpusat Publish-Subscribe, sistem mikrokontroler NodeMCU ESP8266 terbebas dari beban manajemen sesi klien individual berkat kehadiran MQTT Broker sebagai pengelola perutean pesan yang tangguh, ringan, dan berskala besar.

Penggunaan format JSON melalui pustaka ArduinoJson memampukan pengemasan variabel sensor lingkungan (DHT11/DHT22) menjadi paket telemetri terstruktur yang mudah diproses oleh platform *cloud* dan dasbor industri (*SCADA/IoT Platform*). Dengan memadukan mekanisme pengiriman periodik *non-blocking*, penanganan instruksi kendali LED asinkron melalui *callback*, proteksi keselamatan lokal (*Closed-Loop Rule Engine*), serta jaminan keandalan status melalui *Last Will and Testament* (LWT), mahasiswa telah menguasai kompetensi esensial dalam membangun simpul IoT kelas industri yang mandiri, adaptif, dan siap diintegrasikan dengan arsitektur komputasi awan**.**